

Iteration and Loop Control Structures in C++

[DEF] Definition to Remember

Iteration structures in C++ automate repetitive algorithms through for, while, do-while, and range-based for loops, optimized by compiler branch prediction.

1. Concept Overview & Fundamentals

Iteration and Loop Control Structures in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

* Ze

2. Technical Architecture & Memory Mechanics

C++ Runtime Execution Architecture



Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>**std::cin` / `std::cout`**</code>	<code>`scanf` / `printf`</code>	Type-safe, extensible for custom class	Buffered I/O (optimizable via <code>`ios::sy</code>
<code>**References (`Type&`)**</code>	Pointers (<code>`Type*`</code>)	Non-nullable, clean syntax without der	Direct alias; zero runtime overhead
<code>**std::vector<T>`**</code>	Raw Dynamic Array (<code>`malloc`</code>)	Automatic resizing, bounds safety, mem	Amortized <code>**O(1)**</code> push_back, contiguo
<code>**`nullptr`**</code>	<code>`NULL` / `0`</code>	Strongly typed pointer literal prevent	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Fast I/O and standard namespace configuration
int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::cout << "Executing C++ Module: Iteration & Loop Control Structure" << "\n";

    return 0;
}
```

4. Real-World Applications & System Engineering

- * Quantitative Finance & HFT: High-frequency trading platforms use modern C++ for microsecond-level execution.

* Ga

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: ArmstrongNumber

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Pro
 */

/*
 * ===
 * COM
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 *
 *
 *
 *
 *
 *
 */

#include
```

```
#include  
  
// Use  
using  
  
//  
int ma  
in  
  
//  
co  
//  
ci  
  
on  
  
in  
//  
wh  
  
}  
  
te  
//  
wh  
  
}  
  
//  
if  
  
//  
el  
  
//  
re  
  
}
```

Problem 2: EvenPrint

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Pro  
 */  
  
/*  
 * ==  
 * CON  
 * ==  
 * 1.  
 *  
 */
```

```
* 2.
*
step.
*
* 3.
*
*
* 4
* *
*
*
*
* ==
*/

#include

// Use
using

//
int ma
in
//
co
//
ci

//
co
//
fo

}
//
co

//
re
}
```

Problem 3: Factorial

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
* Que
* Wri
*/

/*
* ==
* CON
* ==
* 1.
```

```
*
*
* 2.
*
*
* 3.
*
*
* 4.
* *
*
*
*
* ==
*/

#include

// Use
using

long l
    //
    if
    lo
    //
    fo

}
//
re

}

//
int ma
in
//
co
//
ci

lo
//
if

//
}

}

//
re

}
```

Problem 4: Fibonacci

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Pro
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */

#include

// Use
using

//
int ma
in
//
co
//
ci
lo

//
co
//
fo
```

```
}  
//  
co  
  
//  
re  
}
```

Problem 5: GCD

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
int fi  
//  
wh  
  
}  
//  
re  
}  
  
//  
int ma  
in  
//  
co  
//
```

```
ci
//
cc
//
re
}
```

Problem 6: InfiniteLoop

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Pro
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */

#include

// Use
using

//
int ma
in
//
cc

//
wh
```

```
}  
//  
re  
}
```

Problem 7: LCM

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
int gc  
//  
re  
}  
  
//  
int ma  
in  
//  
co  
//  
ci
```

```
in
//
cc
//
re
}
```

Problem 8: Multiplication

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Dev
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */

#include

// Use
using

//
int ma
in
//
cc
//
ci

//
cc
//
fo

}
```

```
//  
re  
}
```

Problem 9: OddSum

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 * 3.  
 *  
 * 4.  
 * *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
//  
co  
//  
ci  
  
//  
fo  
  
}  
  
//  
co  
  
//  
re  
}
```

Problem 10: PalindromeNumber

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Pro
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */

#include

// Use
using

//
int ma
in

//
co
//
ci

n

//
wh

}

//
if

//
el
```

```
//  
re  
}
```

Problem 11: PatternPrint

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
//  
co  
//  
ci  
  
//  
fo
```

```
}  
//  
re  
}
```

Problem 12: PrimeNumber

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Pro  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 * 3.  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
bo  
  
//  
co  
//  
ci  
  
//  
if  
  
//  
}
```

```
}  
  
//  
if  
  
//  
el  
  
//  
re  
}
```

Problem 13: ReverseNumber

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Pro  
 */  
  
/*  
 * ==  
 * CON  
 * ==  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 * reduce  
 *  
 * ==  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma
```

```
in
//
cc
//
ci

in
//
wh

}

//
cc

//
re
}
```

Problem 14: SumOfDigit

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ===
 */

#include

// Use
using

//
int ma
in
```

```
//  
cc  
//  
ci  
  
in  
//  
wh  
  
}  
  
//  
cc  
  
//  
re  
}
```

Problem 15: SumPositive

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Pro  
 */  
  
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
  
//
```

```
cc
//
fo

}

//
cc

//
re
}
```

Problem 16: even numbers continue

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ===
 */

#include

// Use
using

//
int ma
//
cc
```

```
//  
fo  
  
}  
//  
co  
  
//  
re  
}
```

Problem 17: multiplication table

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 *====  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
//  
co  
//  
ci  
//
```

```
fo
}
//
re
}
```

Problem 18: positive number

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */

#include

// Use
using

//
int ma
in
do

//
}

//
co

//
```

```
re  
}
```

Problem 19: prime number

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 * 3.  
 *  
 * 4.  
 * *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
bo  
  
//  
co  
//  
ci  
  
//  
if  
  
//  
}
```

```
}  
  
//  
if  
  
//  
}  
  
}  
  
//  
re  
}
```

Problem 20: square break loop

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Wri  
 */  
  
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
//  
co  
  
//  
wh
```

```
}  
//  
re  
}
```

Problem 21: sum positive continue

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
  
//
```

```
cc
//
ci

//
fo

}

//
cc

//
re
}
```

Problem 22: sum until zero

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Dev
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include
```

```
// Use
using

//
int ma
do

//
co

do

//
}

//
co

//
re
}
```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. Loop bounds and off-by-one errors: Always verify boundary conditions ($i < N$ vs $i \leq N$).
2. Fibonacci, Prime checks, and GCD/LCM utilize loop state accumulation.
3. Range-based for loop: `for (const auto& elem : container)` provides safe iteration.
4. Time complexity of single loops is $O(N)$; nested double loops evaluate to $O(N^2)$.
5. Loop invariant maintenance ensures correctness of iterative algorithms.

[RECALL] Quick Recall

Entry Test (for/while) -> Iteration Block -> Exit Test (do-while) -> Range Loop -> $O(N)$ Bounds